Creating an LLVM Runtime

EuroLLVM 2026

Michael Kruse

Advanced Micro Devices GmbH

14th April, 2026

Outline

Introduction

Let's Create a Runtime

And They Lived Happily...

Legacy Flang Runtime

New Flang-RT

Remaining Work

AMD
together we advance_

Outline

Introduction
- Why This Tutorial?
- History
- CMake Basics
- Rationale
- Goal

Let's Create a Runtime

And They Lived Happily...

Legacy Flang Runtime

New Flang-RT

Remaining Work

AMD
together we advance_

Why This Tutorial

[redacted] 1 hour ago Member

After this change, it's no longer possible to run openmp tests in a non-bootstrapping build (i.e. without building clang at the same time).

Previously it was possible to run the tests in a standalone build, by building the `FileCheck` and `not` binaries and pointing at them with `OPENMP_LLVM_TOOLS_DIR` - like I did at mstorsjo/llvm-mingw@ `20260210` /.github/workflows/build.yml#L583-L612.

Now after this change (or before, when doing a runtimes build), running tests only works if there's a `FileCheck` target already in the build, i.e. if doing a bootstrapping build:

llvm-project/openmp/cmake/OpenMPTesting.cmake
Lines 52 to 74 in c6425aa

```
52    if (${OPENMP_STANDALONE_BUILD})
53      find_standalone_test_dependencies()
54
55      # Set lit arguments.
56      set(DEFAULT_LIT_ARGS "-sv --show-unsupported --show-xfail")
57      if (MSVC OR XCODE)
58        set(DEFAULT_LIT_ARGS "${DEFAULT_LIT_ARGS} --no-progress-bar")
59      endif()
60      if ("${CMAKE_SYSTEM_NAME}" MATCHES "AIX")
61        set(DEFAULT_LIT_ARGS "${DEFAULT_LIT_ARGS} --time-tests --timeout=3000")
62      endif()
63      set(OPENMP_LIT_ARGS "${DEFAULT_LIT_ARGS}" CACHE STRING "Options for lit.")
```


AMD
together we advance_

Why This Tutorial

[redacted]

[redacted] on Dec 8, 2025 • edited

Yep, just for context from the Rust side. We have a lot of builders in CI, but the slowest one we have is the x86 dist builder, which we use to e.g. create compiler artifacts. We already build LLVM there around 4 times due to PGO and Bolt, so we really try hard not to make it any slower, since we are already at ~3.5 hrs, and we run this CI before any merge into main.

One of the compromises there is that we have two checkouts of LLVM - once as a standalone llvm/llvm-project build (A) and once as a submodule of rustc in `src/tools/llvm-project`. We use build A to compile C/C++ code, and keep rebuilding the second checkout (B) during bootstrapping.
We only build clang as part of our LLVM Build A. However, rustc is based on LLVM B, and as such we'd also want the openmp and offload libraries to also be part of our LLVM Build B, which does not build the in-tree clang to save compile times.

We've tried for quite a bit to get the runtimes to build despite no in-tree clang or lld in LLVM B, and failed, I'm happy to share all the issues I ran into there. A standalone build instead allows me to use clang from build A to build offload+openmp from checkout B, that's compatible with what we have in rustc. So until that is fixed, standalone builds are likely the only builds that our infra team would be ok with.

AMD
together we advance_

Why This Tutorial

[redacted] on Dec 8, 2025 • edited

[redacted] 1 hour ago • edited Contributor

We were using the OpenMP standalone build mode in order to build the static libomp.a runtime library which cannot be build together with the shared libomp.so (there is no FLANG_RT_ENABLE_SHARED and FLANG_RT_ENABLE_STATIC equivalent for libomp, the LIBOMP_ENABLE_SHARED flag is binary: one can get either shared or static library, but never both in the same build).

The OpenMP standalone build mode has been removed by llvm/llvm-project#182022

This complicates our situation. Considering almost nonexistent demand for the static OpenMP runtime library, it is not worth pursuing it.

AMD
together we advance_

Why This Tutorial

[redacted] on Dec 8, 2025 • edited

[redacted] 1 hour ago • edited Contributor

[redacted] on Jan 7 • edited Member

That comes with two drawbacks: 1) checking out `llvm` alone (I'm not talking about `llvm-project`) takes a lot of time. 2) the way to pass CMake arguments to runtime build is extremely user unfriendly. If these two can't be improved, I strongly object the removal of standalone build.

AMD
together we advance_

Why This Tutorial

[redacted] on Dec 8, 2025 • edited

[redacted] 1 hour ago • edited

Member

[redacted] on Dec 8, 2025

Rust colleagues rely on standalone builds for their offloading extensions (see rust-lang/rust#148671).

CC @ZuseZ4

AMD
together we advance_

Why This Tutorial

[redacted] on Dec 8, 2025 • edited

[redacted] 1 hour ago • edited

Member

[redacted] on Jan 7

I don't have a build tree. That's the whole point of building standalone. And the installed binary tree doesn't work.

AMD
together we advance_

Why This Tutorial

[redacted] on Dec 8, 2025 • edited

[redacted] on Jul 26, 2025 Member

It is definitely not the same!
For standalone build, I used to not needing the llvm-project tree. I could build libomp by just downloading the OpenMP release tarball: `llvmorg-21.1.0-rc1` openmp-21.1.0-rc1.src.tar.xz (download)

I just realized that this workflow got bricked by #62240

AMD
together we advance_

History

- Initially, each subproject had its own SVN repository
- No difference between projects and runtimes
- Step 1: `LLVM_ENABLE_PROJECTS`
- Step 2: Sort into tools/<subproject> and runtimes/<subproject> directories
- Step 3: `LLVM_ENABLE_RUNTIMES`
- `https://github.com/llvm/llvm-project/pull/170693#issuecomment-3720262808`
- Original proposal: `https://lists.llvm.org/pipermail/llvm-dev/2020-October/145997.html`

AMD
together we advance_

CMake Basics

- Do not use `set(VAR_NAME)`

AMD
together we advance_

CMake Basics

- Do not use `set(VAR_NAME)`
- Do not use `if (${VAR_NAME})`

AMD
together we advance_

Rationale

- Compile runtimes using just-built Clang
- Compile runtimes for each target platform
  - Cross-compilation
- Avoid combinatorial explosion of build modes

AMD
together we advance_

Purpose

Our Runtime

```
void _dublin_hello_world() {
  puts("Dia daoibh, a dhomhain!\n");
}
```

- libdublin.a/.so
- Linux Only
  - No Windows/MacOS/AIX/...

AMD
together we advance_

Outline

Introduction

Let's Create a Runtime
- Step 0: Setup
- Step 1: Register with the LLVM Build System
- Step 2: Build a Library Artifact
- Step 3
- Step 4
- Step 5
- Step 6
- Step 7
- Step 8
- Step 9
- Step 10
- Step 11
- Step 12
- Step 13
- Step 14
- Step 15

And They Lived Happily...

Legacy Flang Runtime

New Flang-RT

Remaining Work

AMD
together we advance_

Setup

- Code found at `https://github.com/meinersbur/llvm-dublin`
- One branch per step
  - Starting point is `https://github.com/Meinersbur/llvm-dublin/tree/dublin-0-main`
- Changes as PR to previous branch

AMD
together we advance_

Step 1: Register with the LLVM Build System

▶

AMD
together we advance_

Step 2: Build a Library Artifact

▶

AMD
together we advance_

Add an Example that uses the Library

▶

AMD
together we advance_

Build Modes

- Bootstrapping runtimes
- Runtimes standalone
- Runtimes standalone with an LLVM install dir
- Runtimes standalone without an LLVM build

AMD
together we advance_

Artifact Output Location

▶

AMD
together we advance_

Shared and Static Libraries

- Add a Configuration option
-

AMD
together we advance_

Installation

▶

AMD
together we advance_

Add a Regression Test

▶

AMD
together we advance_

Add a Unittest

▶

AMD
together we advance_

Sphinx Docs

▶

AMD
together we advance_

Doxygen Docs

▶

AMD
together we advance_

Cross-Compilation

▶

AMD
together we advance_

Depending on Other LLVM Libraries

▶

AMD
together we advance_

Using C++ in a Runtime

▶

AMD
together we advance_

Accelerator Offloading

▶

AMD
together we advance_

Outline

Introduction

Let's Create a Runtime

And They Lived Happily...
- More Topics
- Build System Improvements
- Conclusion
- Usage
- The Mechanism
- Advanced Options

Legacy Flang Runtime

New Flang-RT

Remaining Work

AMD
together we advance_

Uncovered Topics

- Platform Support: Windows/MacOS/AIX/Fuchsia/...
- Multilib
- ...

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target
- Fat libraries

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target
- Fat libraries
- Get rid of `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR`

AMD
together we advance_

Future Improvements

- Inform LLVM developers and distributions
- `LLVM_ENABLE_RUNTIMES` Documentation
- Propagate `RUNTIMES_*` options
- Unify the CMake code of all the runtimes
  - Use `libdublin` as a template?
- Make Less libcxx-centric
- Writing runtimes in C++
- compiler-rt builtins special handling
- GPUs like any other target
- Fat libraries
- Get rid of `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR`
- ...

AMD
together we advance_

Conclusion

- Now you know how `LLVM_ENABLE_RUNTIMES` works
- Use the Knowledge Wisely
- I currently cannot recommend working on improving the system, waste of your time

AMD
together we advance_

LLVM_ENABLE_RUNTIMES?

Bootstrapping-Runtimes build

```
cmake ../llvm -GNinja                                          \
  "-DLLVM_ENABLE_PROJECTS=clang;lld;polly"                     \
  "-DLLVM_ENABLE_RUNTIMES=compiler-rt;libc;libcxx;openmp"
```

LLVM_ENABLE_PROJECTS

- Compiled using host compiler (e.g. GCC)
- Same CMake build-dir as LLVM itself
- Intended to run on the host architecture

LLVM_ENABLE_RUNTIMES

- Compiled using just-built Clang
- Use separate CMake build-dir nested inside LLVM build-dir
- Intended to be used by binaries compiled by Clang
  - Can be a different architecture (cross-compilation)

AMD
together we advance_

How does it work?

CMake step

LLVM_ENABLE_PROJECTS

1. For each enabled project,
   `add_subdirectory(llvm-sourcedir/<project>)`

LLVM_ENABLE_RUNTIMES

1. Add build targets:
   `runtimes`, `install-runtimes`, `<runtime>`, `check-<runtime>`, `install-<runtime>`, …
2. For each target architecture,

```
llvm_ExternalProject_Add(runtimes ...)
```

   which executes

```
cmake -GNinja                                          \
  -S llvm-sourcedir/runtimes                           \
  -B llvm-builddir/runtimes/runtimes-bins              \
  -DLLVM_BINARY_DIR=llvm-builddir                      \
  -DCMAKE_{C,CXX}_COMPILER=llvm-builddir/bin/clang{++} \
  -DCMAKE_{C,CXX}_COMPILER_WORKS=YES                   \
  "-DLLVM_ENABLE_RUNTIMES=<runtimes>"
```

   2.1 `find_package(LLVM)`, `find_package(Clang)`
   2.2 Find tools such as `llvm-lit`, `FileCheck` in `llvm-builddir`
   2.3 For each enabled runtime,
   `add_subdirectory(llvm-sourcedir/<runtime>)`

AMD
together we advance_

How does it work?

Ninja step

LLVM_ENABLE_PROJECTS: `ninja <project>`

1. CMake ensured all necessary dependencies

LLVM_ENABLE_RUNTIMES: `ninja <runtime>`

1. Build dependencies such as `clang`, `FileCheck`, etc
2. Run configure step for runtimes
3. Build dependencies for runtimes
4. Execute

```
ninja -C llvm-builddir/runtimes/runtime-bins <runtime>
```

   4.1 Build selected runtime

AMD
together we advance_

Runtime Options

Pass Options to Runtimes Build

```
cmake ...                                                    \
  -DCMAKE_CXX_FLAGS=-fmax-errors=1                           \
  "-DRUNTIMES_CMAKE_ARGS=-DCMAKE_CXX_FLAGS=-ferror-limit=1"
```

- `-fmax-errors=1` is for gcc
- `-ferror-limit=1` is for clang

Multiarch & Pass Arch-specific Options

```
cmake ...                                                    \
  "-DLLVM_RUNTIME_TARGETS=default;aarch64-linux-gnu"         \
  "-DRUNTIMES_aarch64-linux-gnu_CMAKE_CXX_FLAGS=-march=cortex-a57"
```

- Will create one builddir per target

AMD
together we advance_

Standalone-Runtimes build

CMake step

```
cmake -GNinja llvm-srcdir/runtimes      \
  -DLLVM_BINARY_DIR=llvm-builddir       \
  "-DLLVM_ENABLE_RUNTIMES=<runtimes>"
```

- Projects (LLVM, Clang, ...) compiled separately
- Uses default C/C++ compiler (e.g. gcc)

Ninja step

```
ninja <runtime>
ninja check-<runtime>
```


AMD
together we advance_

Outline

Introduction

Let's Create a Runtime

And They Lived Happily...

Legacy Flang Runtime
- Usage
- The Problem

New Flang-RT

Remaining Work

AMD
together we advance_

Legacy Flang Runtime

`flang/runtime/CMakeLists.txt`

In-Tree build

- Like a `LLVM_ENABLE_RUNTIMES` build:

```
add_subdirectory(runtime)
```

Standalone build

```
cmake llvm-srcdir/flang/runtime \
  -DLLVM_DIR=...                \
  -DCLANG_DIR=...               \
  -DMLIR_DIR=...

cmake llvm-srcdir/flang/lib/Decimal \
  -DLLVM_DIR=...                    \
  -DCLANG_DIR=...                   \
  -DMLIR_DIR=...
```


AMD
together we advance_

What is the Problem?

- Inconsistent with other LLVM runtimes
- For cross-compilation targets, must compile each target in standalone build
- GPU offloading: Build auxiliary target runtime separately
  - As done for openmp-offload, libc, compiler-rt, libcxx, ...
- Standalone build does not include `iso_fortran_env_impl.f90`
- Source code shared with Flang and Runtime
  - ABI assumed to be the same
  - Compile code and runtime code have different requirements
    E.g. runtime code must not link to C++ standard library
  - No clear separation which file belongs where
- Compiled binary shared with Flang and Runtime
  - Runtime built with different flags
    E.g. `-fno-lto`

AMD
together we advance_

Outline

Introduction

Let's Create a Runtime

And They Lived Happily...

Legacy Flang Runtime

New Flang-RT
- Usage
- The Difficulties
- The Changes

Remaining Work

AMD
together we advance_

Building Flang-RT

Bootstrapping-Runtimes build

```
cmake  -GNinja ../llvm                          \
  "-DLLVM_ENABLE_PROJECTS=clang;mlir;flang" \
  "-DLLVM_ENABLE_RUNTIMES=flang-rt"
```

Standalone-Runtimes build

```
cmake -GNinja llvm-srcdir/runtimes                  \
  "-DLLVM_ENABLE_RUNTIMES=flang-rt"                 \
  -DLLVM_BINARY_DIR=llvm-builddir                   \
  -DCMAKE_Fortran_COMPILER=llvm-builddir/bin/flang \
  -DCMAKE_Fortran_COMPILER_WORKS=YES
```

- Flang must built from the same git SHA1
  No ABI contract
- CMAKE_Fortran_COMPILER_WORKS because flang before the runtime is available cannot produce executables

AMD
together we advance_

Things that Must Continue Working

- Shared library
- Quad-precision `math.h` support
  - gcc `libquadmath`
  - Native `sizeof(long double) == 16` with `libm`
  - f128 suffix functions (like `sinf128`) in `libm`
- Conditional `REAL(16)` support in Flang
- Unittests
  - GTest and "non-gtest" testing framework
- Windows static `.lib`
  - LLVM emits libgcc-ABI function calls, requires `clang_rt.builtins.lib` at link-time
  - msvc ships `clang_rt.builtins-x86_64.a`, but not used by the driver (anymore)
- Experimental OpenMP-offload build
- Experimental CUDA build
  - With `clang -x cuda` and `nvcc`

AMD
together we advance_

Library Names

Old Library Names

- `libFortranRuntime{.a,.so}`
- `libFortranDecimal{.a,.so}`
- `libFortranFloat128Math.a`
- `libCufRuntime_cuda_${version}{.a,.so}`

New Library Names

- `libflang_rt.runtime{.a,.so}`
- `libflang_rt.quadmath.a`
- `libflang_rt.cuda_${version}{.a,.so}`

- Same scheme as Compiler-RT: `libclang_rt.<component>{.a,.so}`
- `libFortranDecimal` integrated into `libflang_rt.runtime`
  - Decided by RFC
  - `libFortranCommon` also used by Flang
  - Made `flang` depend on `libcudart.so`

AMD
together we advance_

The Big Move

Principles

- Split some headers into a compiler- and a runtime part
- Definitions to `flang-rt/lib/$component/*.cpp`
- Non-private headers to `flang-rt/include/flang-rt/$component/*.h`
- Files used by both, Flang (the compiler) and Flang-RT (the runtime), remain in `flang/`
- Move "Common" files only used by Flang (the compiler) to `Support`
  - Remaining shared components: `FortranDecimal`, `FortranCommon` (header-only), `FortranRuntime` (header-only), `FortranTesting`

Old	New
`flang/runtime/*.h`	`flang-rt/include/runtime/*.h`
`flang/runtime/*.cxx`	`flang-rt/lib/runtime/*.cpp`
`flang/runtime/Float128Math/*`	`flang-rt/lib/quadmath/*`
`flang/runtime/CUDA/*`	`flang-rt/lib/cuda/*`
`flang/include/flang/Runtime/*.h`	`flang/include/flang/Runtime/*.h`
`flang/include/flang/Common/*.h`[1]	`flang/include/flang/Support/*.h`
`flang/unittests/Evaluate/{fp-}testing.h`	`flang/include/flang/Testing/*.h`
`flang/lib/Common/*.cpp`	`flang/lib/Support/*.cpp`
`flang/unittests/Evaluate/{fp-}testing.cpp`	`flang/lib/Testing/*.cpp`
`flang/test/**/*`[2]	`flang-rt/test/**/*.cpp`
`flang/unittests/**/*.cpp`[2]	`flang-rt/unittests/**/*.cpp`

AMD
together we advance_

LLVM_ENABLE_PER_TARGET_RUNTIME_DIR

Library Location

- Old Flang Runtime:
  `${CMAKE_INSTALL_PREFIX}/lib/libflang_rt.runtime.a`
  - Clash in multiarch/cross-compile scenarios
- `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR=OFF`:
  `${CMAKE_INSTALL_PREFIX}/lib/clang/$version/lib/$os/libclang_rt.buildins-$arch.a`
  - Windows, Apple, AIX
- `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR=ON`:
  `${CMAKE_INSTALL_PREFIX}/lib/clang/$version/lib/$triple/libclang_rt.builtins.a`
  - Became default for Linux in Clang 19 (Now also BSD, OS390)
  - Assumptions leaking into `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR=OFF` as well 😕

- Only the last supported for flang-rt
  - `${CMAKE_INSTALL_PREFIX}/lib/clang/$version/lib/$triple/libflang_rt.<component>{.a,.so}`
  - `LLVM_ENABLE_PER_TARGET_RUNTIME_DIR` ignored

AMD
together we advance_

Shared Library

- Old scheme: `BUILD_SHARED_LIBS=ON`
  - Requires a second standalone build
- New scheme: Build static+shared library at the same time using *object libraries*
  - Done by (almost) every other runtime

CMake Options

- `FLANG_RT_ENABLE_STATIC`
  - Default: `ON`
- `FLANG_RT_ENABLE_SHARED`
  - Default: `OFF`

    ld prefers `.so` over `.a`, enabling it would be a breaking change

AMD
together we advance_

Experimental GPU Target Support

CUDA

- **clang**: Compile everything with `-x cuda`
- **nvcc**: Treats everything as CUDA source
- Requires libcudac++ (libc++ for CUDA), cannot use `<variant>` or `<optional>`
- Declarations must be annotated with `__host__ __device__`

OpenMP

- Compile everything with `-fopenmp --offload-arch`
- Declarations must be annotated with *`#pragma declare target`*

- Annotations selected with preprocessor macro `RT_API_ATTRS`
- Results in a *fat library*
  - Host and device code in a single file
  - For AMD/OpenMP we would rather compile them separately
    - Multiarch library with device code looked up when launching kernels

AMD
together we advance_

Outline

Introduction

Let's Create a Runtime

And They Lived Happily...

Legacy Flang Runtime

New Flang-RT

Remaining Work
TODOs

AMD
together we advance_

To Do Items

- Flang is not (yet) a cross-compiler
  Sometimes assumes ABI of host platform, e.g. `sizeof(long)`
- Compile builtin modules in the runtimes build using CMake
  - OpenMP's modules as well
  - Per-target modules
- Flang's Quadmath support must not depend on LLVM build environment
- Multilib support
- Shared library location
- Library versioning

AMD
together we advance_

AMD